

Efficient intrusion detection via heterogeneous graph attention networks and parallel provenance analysis

Lin Wu^a, Yu-Lai Xie^{a,e,*}, Shi-Xun Zhao^b, Pan Zhou^a, Dan Feng^{b,c}, Avani Wildani^d, Ya-Feng Wu^a

^a School of Cyber Science and Engineering, Huazhong University of Science and Technology, 430074, Wuhan, China

^b School of Computer Science and Technology, Huazhong University of Science and Technology, 430074, Wuhan, China

^c Wuhan National Laboratory for Optoelectronics, Huazhong University of Science and Technology, 430074, Wuhan, China

^d Department of Computer Science, Emory University, Atlanta, 30322, GA, USA

^e Shenzhen Huazhong University of Science and Technology Research Institute, Shenzhen, 518000, China

ARTICLE INFO

Keywords:

Intrusion detection

Provenance

Heterogeneous graph neural network

Parallel

ABSTRACT

In recent years, Advanced Persistent Threats (APTs) have emerged as a significant and pervasive form of cyber attack that uses sophisticated, covert techniques to infiltrate and persist in vulnerable systems, posing a significant threat to businesses and organizations. Recent studies have highlighted the potential of using provenance for APT detection. Provenance is a kind of data that records the history and dependencies of system objects (such as files, processes, and sockets) and is usually converted into a provenance graph for analysis. However, the previous methods have several limitations: (1) The large amount of data generated by long-term APT attacks has a great storage overhead and reduces the analysis efficiency. (2) Requires prior attack knowledge and cannot cope with unknown attacks. (3) It fails to consider the rich semantic information in the provenance graph fully. In this paper, we propose IDS-HGAT, a novel intrusion detection system based on a heterogeneous graph attention network. The system can reduce the number of nodes by preprocessing while retaining the graph structure information. IDS-HGAT can consider the semantic information of different types of nodes and edges and the structure information of the provenance graph, and effectively aggregate the semantic information to build a classification model without constructing a rule base. In order to improve the detection efficiency, IDS-HGAT employs the Stream data type in Redis to build a message queue to support parallel storage and acquisition of provenance data. The experimental results show that IDS-HGAT is better than the existing state-of-the-art methods in terms of precision rate, false alarm rate, and time cost.

1. Introduction

In recent years, network security has become increasingly severe, especially with more large organizations becoming targets of Advanced Persistent Threat (APT) attacks that persist for months or even years. At the end of March 2021, CNA Financial, an American insurance company, was attacked by ransomware and finally paid a ransom of 40 million before regaining control of its network [1]. On June 22 of the same year, hackers sold more than 700 million LinkedIn user data on the dark network platform, and about 92% of LinkedIn users could search for personal information from the leaked database, making the incident the largest data breach in LinkedIn's history [2]. Encryption [3] and access control [4] have been used to enhance computer security for a long time. However, attackers can still exploit various system vulnerabilities (such as unpatched operating systems, programming errors, firewall misconfigurations, and weak passwords)

to compromise computer systems, resulting in sensitive data leakage or system corruption.

Recent works [5–7] have shown that provenance plays a crucial role in APT attack detection and is therefore widely used as a data source for intrusion detection. Provenance records the history of system objects (such as files, processes, and sockets) and reveals the data flows and dependencies between objects. The provenance that conforms to the OPM (Open Provenance Model) [8] can be transformed into a topology graph structure containing three types of nodes and five types of connection edges. Thus the provenance graph can be regarded as a heterogeneous graph. How to process provenance graphs efficiently is a vital part of provenance-based intrusion detection.

Although some works [9–11] have achieved good results by using provenance data for intrusion detection and analysis, there are still limitations: but there are still the following limitations: (1) APT attacks

* Corresponding author.

E-mail addresses: linwu@hust.edu.cn (L. Wu), ylxie@hust.edu.cn (Y. Xie), u201413658@alumni.hust.edu.cn (S. Zhao), panzhou@hust.edu.cn (P. Zhou), dfeng@hust.edu.cn (D. Feng), avani@mathcs.emory.edu (A. Wildani), yafengwu@hust.edu.cn (Y. Wu).

<https://doi.org/10.1016/j.comnet.2025.111552>

Received 28 October 2024; Received in revised form 7 July 2025; Accepted 11 July 2025

Available online 23 July 2025

1389-1286/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

often span long periods, generating massive audit log data during system operation. The common solution is to reduce the data size, but the remaining data still requires considerable effort to analyze [11,12]. (2) Existing methods detect behavior events by matching them with the rules that rely heavily on expert knowledge [13–15]. As the rule set grows, the detection speed decreases. Furthermore, since these rules are designed for specific environments, they tend to introduce biases when applied in real-world scenarios. (3) Statistic-based methods fail to uncover the complex semantic relationships among different entities [5], and they exhibit high false positive rates when dealing with more covert and sophisticated APT attacks. While deep learning-based methods [16] have shown promising detection performance, they are challenged by substantial memory demands.

To address the above issues, we propose IDS-HGAT, a novel provenance-based intrusion detection system based on heterogeneous graph attention networks. Firstly, the original provenance structure is relatively complex, and the large volume of provenance data is not conducive to training graph neural networks (GNNs) [17]. Therefore, we design a structural-aware provenance preprocessing scheme that removes nodes, node features, and edges with minimal influence on detection, and reconstructs essential connectivity for generating GNN-suitable provenance graphs. Secondly, IDS-HGAT considers different types of nodes and edges simultaneously and can effectively aggregate semantic information to establish a classification model for detection without building a rule base. To improve the detection efficiency, we construct a message queue combined with Redis and perform provenance collection, preprocessing, and intrusion detection for multi-users in parallel. In summary, our research provides a new and effective method for APT attack detection by introducing heterogeneous graph learning, source preprocessing, and Redis-based message queues.

In summary, the contributions of this paper are summarized as follows:

- We propose IDS-HGAT, a novel intrusion detection system based on a hierarchical heterogeneous graph attention network that integrates optimized provenance preprocessing to retain critical structural and semantic information while reducing data volume. This enables efficient and accurate detection by jointly modeling node types and edge dependencies.
- To further improve the detection efficiency, we utilize Redis to construct a message queue to perform provenance collection, preprocessing, and intrusion detection in parallel.
- We evaluate IDS-HGAT on five datasets. The results show that IDS-HGAT has an 8% increase in precision compared to a 7.7% decrease in false alarm rate to state-of-the-art methods on average. Moreover, the time overhead of IDS-HGAT is reduced by about 90.17%–99.32% compared with UNICORN. IDS-HGAT outperforms existing methods overall.

2. Related work

2.1. Provenance-based intrusion detection

Provenance-based intrusion detection methods have been widely studied by researchers. Rule-based detection methods [13–15] rely on prior knowledge of known attack patterns to match and detect these attack behaviors by constructing heuristic rules. For example, HOLMES [13] used Tactics, Techniques, and Procedures (TTPs) rules based on the MITRE ATT&CK framework to match low-level system events with high-level APT attack phases by analyzing system audit logs. These methods can efficiently detect known attacks, but dealing with unknown APT attack patterns is challenging. Statistic-based APT detection methods [18] mainly constructs a statistical model to measure the rarity of entities in the provenance graph or the anomaly of interactive behavior and then carries out anomaly detection. However, rarity does not necessarily equate to abnormal behavior [5], and

statistics-based detection can only reveal superficial features of the provenance graph. It cannot extract deeper semantic information from interactions between different entities. Methods based on deep learning [9,19,20] automatically learn the characteristics of APT attacks by modeling the system's normal behavior or attack pattern. However, the large number of audit logs generated by the enterprise daily makes these methods [9,16] take much time during training. In addition, although the graph-based method [6,19] can extract deep features to learn system entities, it fails to consider the deep semantic information in different types of entity interactions. TDLens [21] provides an empirical evaluation framework for existing threat detection methods based on provenance graphs. However, the core goal of TDLens is to compare and evaluate existing methods horizontally and vertically rather than proposing new detection techniques. In contrast, our method proposes a novel intrusion detection model that optimizes feature selection and embedding techniques to improve detection performance. Even though FLASH [22] uses Word2Vec and Graph Neural Network (GNN) to extract semantic and contextual information, it cannot process heterogeneous relationships. It cannot effectively handle the multiple types of nodes and edges present in the provenance graph. RT-APT [23] is an APT detection method for large-scale provenance graphs, which effectively aggregates node context information using WL subtree accounting. However, it relies on the sorting and compression of node labels, which may result in the loss of some subtle semantic differences.

Compared with the above methods, IDS-HGAT can simultaneously capture rich relationships between different types of entities and more comprehensively model complex behavioral patterns in APT attacks. This method performs well in the face of known attack patterns and better addresses the challenges of unknown attacks. In addition, IDS-HGAT efficiently processes large-scale provenance graphs through effective storage and parallel detection, avoiding the high computational overhead problem of statistical or traditional deep learning methods in large-scale log data processing.

2.2. Data reduction

The current advanced persistent threat attack has many phases and a long duration. The amount of provenance data collected will be very large, bringing great challenges to attack detection and forensic analysis. Most methods mainly analyze the dependencies of the system audit log. These methods assume that an output operation depends on all previous inputs [24,25], which is inaccurate. For example, the dependencies in an attack provenance graph contain many benign dependencies, but often only a small number of dependencies are related to attacks [26]. Therefore, it has been proposed to divide long-running programs into independent execution units, which can eliminate some false dependencies, but requires the support of third-party tools [27, 28].

Some studies [26,29] improve on the above work, and they prune the provenance graph by ranking low-frequency events. For example, NoDoze [26] counts the frequency of related events in the causal graph and then assigns an anomaly score to each edge to ensure that only anomalous edges in the provenance graph are included. Although it can reduce the size of the graph, it can also produce false positives. ALchemist [30] parses and analyzes important events in the log and then reconstructs the execution unit structure to analyze dependencies more accurately.

Some methods focus on removing unnecessary nodes and edges without losing important information. LogGC [31] deleted some temporary files in the data. However, if the temporary files are related to some network sockets, it cannot catch the data overflow attack made by the attacker. NodeMerge is an online template-based method [12] for system data reduction that merges read-only events during program initialization, ignoring write-heavy hosts. The method proposed by Hosian et al. [32] removes duplicate edges on the same host and duplicate events across hosts. Although these methods improve storage efficiency,

some information is inevitably lost. Fei et al. proposed SEAL [11], a general and more efficient compression method for causal analysis. They developed a QFC framework that is superior to the above three methods in compression, event handling, and operations on duplicate edges.

In order to reduce the size of the provenance graph, previous methods prune irrelevant nodes and wrong dependencies as much as possible to reconstruct attack subgraphs or perform forensic analysis. However, when a node is deleted, the edges connected to it are also deleted, which will destroy the provenance graph structure. Our method connects parent and child nodes using a specific edge type, avoiding information loss, which was not taken into account by previous methods. Instead of analyzing huge dependencies, we focus on their frequencies as part of the features, which will not cause much analysis burden.

2.3. Graph neural network

Graph Neural Network (GNNs) is a deep learning method used to process graphs and has been widely used in various applications [33–35]. It is often closely linked to graph embedding learning, it aims to learn low-dimensional vector representations of network nodes while retaining network topology and node content information, and then proceed to downstream tasks. Recently, attention mechanisms in graph learning have been extensively studied, which can process variable-size data and focus on the most significant parts of the data.

Velickovic et al. [36] proposed the Graph Attention Network (GAN), which uses the attention mechanism for homogeneous graphs that include only one type of node or connection. In 2019, Wang et al. [37] first proposed a heterogeneous graph neural network (HAN) based on attention mechanism that can be widely used in heterogeneous graph analysis. HAN follows the classic heterogeneous graph neural network architecture (node-level aggregation and architecture-level aggregation). In order to better implement the hierarchical aggregation function, HAN first uses node-level attention to learn the embedding of each node based on each meta-path and then aggregates the embedding of nodes based on each meta-path through architecture-level attention to obtain the final node representation. In the same year, Zhang et al. [38] proposed HetGNN. Unlike HAN, HetGNN does not consider node-level attention but uses LSTM [39] to aggregate node neighbors under a certain relationship and update node representation.

The above neural network can achieve better results in some application scenarios (such as node classification and link prediction), but when analyzing attacks, we should consider both the node information and the dependencies between nodes. It is inaccurate to identify intrusions only by analyzing nodes using the above methods [17].

3. Background and motivation

In this section, we first introduce a provenance overview. We then describe provenance-based intrusion detection and its challenges with a motivating example. Finally, we briefly analyze the solutions and describe the threat model.

3.1. Provenance overview

Provenance is metadata that records the history of system objects (such as files, processes, and sockets) and collects the state and relationships between objects in the system. Provenance can be represented as a directed acyclic graph. System objects are nodes in the graph, and each node has its own attributes. Nodes are connected by edges, which represent the data flow or control flow between connected objects. Common provenance collection systems include PASS [40], SPADE [41] and CamFlow [42].

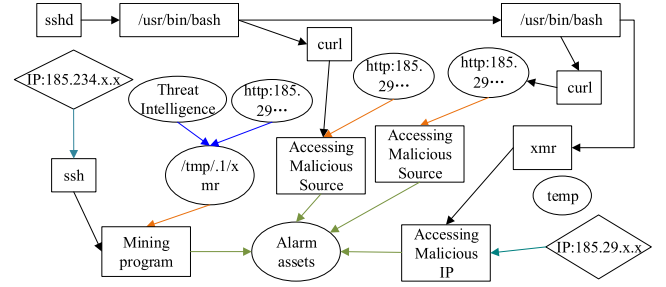


Fig. 1. An example of partial provenance graph of worm propagation. Rectangles denote processes, oval-shaped denote files and diamonds denote network objects. The different colored edges in the figure represent different dependencies.

3.2. Provenance-based intrusion detection and challenges

Scenario: Consider the following worm propagation scenario. A worm propagation source (for example, IP 185.234.x.x) is brute-forced and logged into the server through ssh. The mining program is downloaded from the remote end by executing the curl command through bash, and then the mining program is run on the server.

Provenance reveals the generation process of data objects, through which we can quickly understand the information of objects and the dependencies between them. The traditional provenance-based intrusion detection method establishes a rule database representing the dependencies among files, processes, and agents. When there is a new process activity in the system, whether an intrusion has occurred can be analyzed by comparing it with the dependency relationship in the rule database. The research of provenance in intrusion detection mainly improves the accuracy and efficiency of intrusion detection.

Challenges: But the current provenance-based intrusion detection methods exist the following problems:

(a) The original provenance data contains information that is unrelated to intrusion detection. For example, the temporary file *temp* in Fig. 1 is generally less likely to store intrusion information as an isolated node (it may just be a bridge to pass information between entities); In addition, the amount of original provenance data is too large, which brings a great burden to data storage. Fig. 1 is only part of the provenance graph in worm propagation. In practice, the provenance graph will become huge with time.

(b) Most existing studies [14,43] mainly rely on manually designed heuristic rules. However, the completeness of the rule base directly affects the coverage and effectiveness of detection. With the expansion of the rule base, the detection system needs to match each data point with the rule base one by one in the execution process, resulting in a significant decrease in detection efficiency. Moreover, rule-based intrusion detection methods exhibit obvious limitations when facing unknown complex attack patterns.

(c) Statistic-based methods primarily rely on historical data or existing attack patterns, but they struggle to capture complex, multi-step APT attacks through simple statistical features. Additionally, these methods [6,18] often lack an understanding of the semantic information required for detecting sophisticated attacks. They are prone to misjudgment of legitimate but uncommon behaviors, resulting in a high false positive rate. Deep learning-based methods [16,19] encounter memory overhead issues when dealing with large-scale data. Moreover, some methods [19,43] fail to fully capture the different entities and relationships within provenance graphs, neglecting critical information, which impacts detection performance.

3.3. Solution analysis

3.3.1. Provenance preprocessing

In response to the problem above (a), provenance can be pre-processed to simplify the provenance data. Preprocessing needs to

delete irrelevant information to ensure that the provenance graph is compressed as much as possible without losing detection information and without destroying the structure of the provenance graph. First we delete some irrelevant nodes. For example, some nodes that do not contain the NAME attribute are all temporary objects that only describe the transformation of information between different entities (e.g., files or processes) and are unlikely to store intrusion information. In order to reduce the impact of node deletion on the provenance graph structure, the parent node and child node of the deleted node are connected with edges of corresponding types. Then, we remove attributes in the provenance node that are not relevant for intrusion detection. For example, although the NAME and PID attributes of the node can play the role of identifying the node, when constructing the source graph object, IDS-HGAT preprocessing algorithm stores the node characteristics of each source graph in a corresponding list. The subscript can uniquely identify the node. Therefore, the NAME attribute and the PID attribute can be deleted. Finally, provenance is constructed into a format-specific dictionary and stored in the database.

3.3.2. IDS-HGAT intrusion detection

For problem (b), we need to develop a scheme that comprehensively considers the semantic information of provenance graph nodes and graph structure information. The graph attention network is suitable for processing graph data in non-Euclidean space [17,44], such as provenance graph. However, only a few studies have applied it to provenance data. Therefore, we analyze the feasibility of using a graph attention network for provenance-based intrusion detection and propose an intrusion detection scheme based on a heterogeneous graph attention network.

3.3.3. Parallel intrusion detection based on redis message queue

Aiming at problem (c), the paper extracts latent patterns from training datasets and establishes a detection model without building a rule base. We propose a parallel execution strategy for provenance collection, preprocessing, and intrusion detection to improve the detection speed.

The attacker's purpose is often to monitor the machine for further actions, ultimately causing damage. In our method, we perform inter-entity information integration (feature extraction) by capturing the inter-dependencies among the entities in the provenance graph and converting them into a numerical vector embedding, where we consider the different types of edges and nodes. The neighbors and their corresponding importance levels more accurately and comprehensively describe the event profiles of the normal and abnormal graphs. Finally, a binary classifier is obtained for anomaly detection.

Compared with previous methods, this paper uses the information associated with processes to learn and embed by aggregating features between entities, considering the heterogeneous information (i.e., different node types and edge types) and corresponding weights between different entities, and then learning model representation. It avoids the problems caused by rules.

Specifically, for a node in the provenance graph, the neighbor nodes connected to it by different types of edges are different, and the importance of different neighbors to the node is also different. We first use the deep neural network of node-level attention to obtain the importance of different types of neighbors to the target node, and then obtain the corresponding weight value. This is not taken into account by previous methods based on provenance graphs. We then use architecture-level attention to learn the importance of different edge types. The learned two-layer attention is aggregated to generate the final node embedding. Node embedding can better capture the complex structure and rich semantic information in the provenance graph. The model finally established and experimentally confirmed the effectiveness of using the attention mechanism, indicating that it has good interpretability for the provenance graph analysis. This also shows that the attention mechanism plays a role in the weight distribution

of the learned content, and what is the important information learned. Moreover, using multi-head attention at the node level can enhance the ability of the model and stabilize the training process, and finally give us satisfactory results.

3.4. Threat model

This paper collects provenance data from kernel-level information through the provenance collection system, so it has the advantages of high reliability and stability. We assume the correctness of the kernel and provenance data. We are committed to attack detection based on heterogeneous graphs such as provenance graphs, which can be detected even if the same behavior is somewhat different in the provenance graphs displayed on different hosts. Even if the provenance graph dynamically grows for long periods, whether the increase is normal or malicious, this paper can identify it because we learn the potential representation of the behavior.

4. Design and implementation

This section presents the design and implementation of the IDS-HGAT intrusion detection method. The method uses the provenance collection system to collect and preprocess provenance data. The data is then stored in a message queue constructed by the Redis database. Finally, IDS-HGAT reads the provenance data in the Redis message queue in parallel and performs intrusion detection.

4.1. Design challenges

The design difficulties of this paper are summarized as follows:

- (1) Preprocessing of provenance graph. On the one hand, the pre-processing provenance graph needs to delete irrelevant information and compress the provenance graph as much as possible. On the other hand, it is critical to avoid losing critical information and make the processed data suitable for IDS-HGAT;
- (2) Which database to use and how to store provenance data to improve the speed of intrusion detection;
- (3) The provenance graph belongs to a heterogeneous graph. What mechanism and method are used to process the provenance graph to extract the information efficiently;
- (4) How to optimize the model's parameters to obtain the best parameter combination;
- (5) How to design an intrusion detection system, including data collection, data preprocessing, data storage, and intrusion detection modules, to improve detection rate and accuracy.

4.2. Overall design

The purpose of this design is to apply graph attention network technology to the field of intrusion detection, increase the speed of intrusion detection through multi-process parallel processing, and improve detection accuracy and efficiency. The overall architecture is shown in Fig. 2, mainly divided into three modules: the parallel provenance collection and preprocessing module, the Redis storage module, and the parallel IDS-HGAT intrusion detection module.

(1) *Parallel Provenance Collection and Preprocessing.* We use the SPADE provenance collection system to collect provenance information and perform preprocessing operations such as filtering and compression. In order to improve the efficiency of provenance collection and preprocessing, we simultaneously start multiple processes for each application that needs to be monitored.

(2) *Redis Storage.* The storage module uses the memory database Redis to store the preprocessed provenance information. Since multi-processes write data to Redis in parallel, and multi-processes in the IDS-HGAT intrusion detection module read data from Redis in parallel, the producer-consumer problem will arise in this scenario. To this

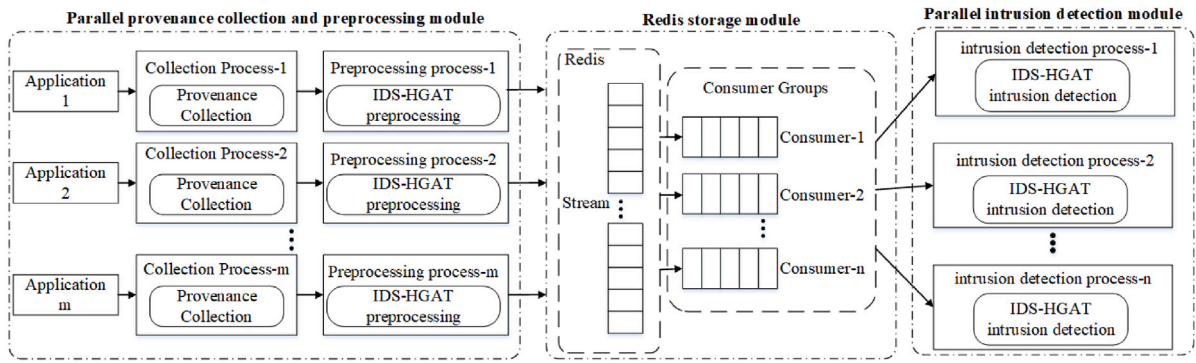


Fig. 2. IDS-HGAT overview.

end, we use the Stream data type in Redis to build a message queue. Several preprocessing processes in the parallel provenance collection and preprocessing module are used as the message queue producers. The provenance graph information obtained by preprocessing is stored in the message queue. Several intrusion detection processes in the parallel detection module are used as message queue consumers. Consumers obtain provenance graph information from the message queue and perform intrusion detection in parallel. Through the Redis storage module, the provenance collection and preprocessing processes do not need to call the interface or method of the intrusion detection process to transfer data and realize decoupling. At the same time, provenance collection and preprocessing can execute other processes after sending the data to the Redis message queue without waiting for the intrusion detection process to process the data, achieving asynchrony. Finally, the detection efficiency of the intrusion detection system has improved.

(3) *IDS-HGAT Intrusion Detection*. HAN [37] inspires the IDS-HGAT method since HAN performs well in dealing with heterogeneous graphs, and the provenance graph is essentially a heterogeneous graph. We transform the node classification in HAN into a provenance graph classification. The process-related edges in the provenance graph are used to distinguish different types of neighbors of nodes. In addition, we add a pooling layer to HAN so that the network can perform graph classification tasks.

The parallel IDS-HGAT intrusion detection module uses the hierarchical attention network IDS-HGAT to classify the provenance graph into normal and abnormal to achieve intrusion detection. Specifically, the module first uses the provenance training set to train the IDS-HGAT network to obtain an IDS-HGAT classifier. Then, the trained classifier is deployed to several intrusion detection processes. The provenance data to be detected is read from the Redis message queue in parallel and classified.

4.3. Parallel provenance collection and preprocessing

Provenance Collection. This paper uses the SPADE provenance collection system to collect provenance information. In order to improve the efficiency of provenance collection, we use multiple SPADE processes to collect dataset in parallel. However, the provenance data collected by the SPADE system will contain a large amount of information unrelated to intrusion detection. How to retain only useful information and efficiently store provenance data is an urgent problem to be solved. Therefore, this paper chooses to preprocess the provenance to delete irrelevant nodes and attributes.

Provenance Preprocessing. The provenance preprocessing mainly includes: (1) Remove invalid nodes. (2) Remove irrelevant node attributes.

(1) *Remove invalid nodes.* First, we delete some temporary nodes that do not contain the name attribute because they only introduce information conversion between different entities (files or processes). It is unlikely to store intrusion information. Second, we delete many

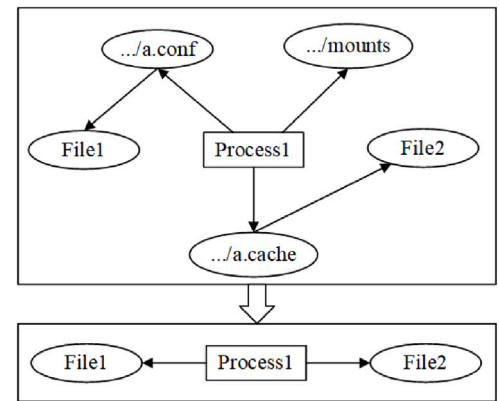


Fig. 3. An example of deleting invalid nodes.

Table 1
Feature vectors of node.

Type	Was Controlled By	Used	Was Generated By	Was Triggered By	Was Derived By
0 (Process)	3	\	2	4	\
1 (Agent)	3	\	\	\	\
2 (Artifact)	\	3	\	\	4

library file nodes and configuration file nodes that are unrelated to detection in the provenance graph. Finally, the temporary cache files unrelated to intrusion detection are also deleted. It is worth noting that process nodes are the critical detection objects, so this method retains all process nodes.

After removing invalid nodes, IDS-HGAT also connects the parent and child nodes of the deleted node through a specific type of edge. For example, the parent node of the deleted node is the process type, and the child node is the file type. WasGeneratedBy type edges will connect the parent and child nodes of this method. Since all process nodes are retained, all the parent and child nodes of the deleted node can be connected by a specific edge type. Fig. 3 shows a typical example of deleting invalid nodes.

(2) *Remove irrelevant node attributes.* Delete many attributes that are not related to intrusion detection. For example, delete the NAME attribute and the PID attribute, although they can play a role in identifying nodes. However, when constructing the provenance graph object, IDS-HGAT stores the node characteristics of each provenance graph in a corresponding list, and the list subscript can uniquely identify the node. IDS-HGAT retains only the TYPE attribute of the node object, which can determine the edge type between two connected nodes without storing the edge type information separately.

Algorithm 1 Delete irrelevant nodes and attributes

Input: *DotFiles* //Provenance File sets
Output: *G* //Provenance graph sets

```

1: for DotFile in DotFiles do
2:   for each line in DotFile do
3:     if pattern of  $line_i$  is vertex then
4:       if vertex is valid then
5:         create new instance  $v_i$ 
6:          $V \leftarrow V \cup V \{v_i\}$ 
7:       end if
8:     end if
9:     if pattern of  $line_i$  is edge then
10:      create new instance  $e_i = v_{child} \rightarrow v_{parent}$ 
11:       $E \leftarrow E \cup E \{e_i\}$ 
12:    end if
13:  end for
14: end for
15: return  $G(V, E)$ 

```

Generally, the structure of the provenance graph that does not contain intrusion is relatively stable, and the frequency of edges related to nodes has a certain regularity [45]. However, the provenance graph structure containing the invasion fluctuates, and the relevant edge frequency of the nodes related to the invasion may have “mutations”. Therefore, IDS-HGAT provenance preprocessing method counts the relevant edge frequency of the node and adds it to its feature vector. The node’s features after preprocessing are shown in Table 1. The feature vector of a node is comprehensively represented by the type of the node and the frequency of five edges starting from the node.

This paper first transforms the provenance information into a provenance graph and deletes irrelevant nodes and attributes. The conversion process is shown in Algorithm 1. After successfully converting it into a provenance graph, it is necessary to construct it into a dataset suitable for the IDS-HGAT intrusion detection method. Algorithm 2 shows the method for reconstructing the provenance graph.

The algorithm flow is mainly divided into four steps.

(1) Based on the provenance graph obtained by Algorithm 2, determine whether the current provenance graph is normal or abnormal. If the current provenance graph is located in the abnormal folder, set the provenance graph label to 1, otherwise set it to 0.

(2) Traverse all the nodes in the provenance graph, create a node object, and add it to the feature vector list *X*. Each item in the list is a six-dimensional vector representing the feature vector of a node.

(3) For all provenance graph nodes, traverse each node’s neighbor nodes. Firstly, the feature vector of the node is updated according to the neighbor type. Secondly, five edge arrays *E0*, *E1*, *E2*, *E3*, and *E4* are constructed according to the different edge types. We have the following description:

E0: A two-dimensional array of size $m0 * 2$. *m0* represents the number of edges of the WasControlldBy type. *E0*[0][0] and *E0*[0][1] represent the source node and target node of the first WasControlldBy edge, respectively.

E1: A two-dimensional array of size $m1 * 2$. *m1* represents the number of edges of the Used type. *E0*[0][0] and *E0*[0][1] represent the source node and target node of the first Used edge, respectively.

E2: A two-dimensional array of size $m2 * 2$. *m2* represents the number of edges of the WasGeneratedBy type. *E0*[0][0] and *E0*[0][1] represent the source node and target node of the first WasGeneratedBy edge, respectively.

E3: A two-dimensional array of size $m3 * 2$. *m3* represents the number of edges of the WasTriggeredBy type. *E0*[0][0] and *E0*[0][1] represent the source node and target node of the first WasTriggeredBy edge, respectively.

Algorithm 2 Reconstructing the provenance graph

Input: $G=\{G_0, G_1, ..., G_i, ..., G_m\}$
 // Provenance graph Sets after removing irrelevant information
Output: $X=\{X_0, X_1, ..., X_i, ..., X_m\}$
 $L=\{L_0, L_1, ..., L_i, ..., L_m\}$
 $E=\{Edge_0, Edge_1, Edge_2, ..., Edge_m\}$
 // X_i represents the feature matrix of G_i
 // L_i represents the label of G_i
 // $Edge_i$ represents the edges of G_i

```

1: if G.Y is normal then
2:   create new instance  $L_i(1)$ 
3: else
4:   create new instance  $L_i(0)$ 
5: end if
6:  $L \leftarrow L_i$ 
7: for source_nodej in  $G_i$  do
8:   create new instance  $X_j(\text{node\_type})$ 
9:    $X \leftarrow X_i$ 
10:  for source_nodej in  $G_i$  do
11:    create new instance  $Edge_j$ 
12:    for node in connected_node of source_node do
13:      for  $Edge_j$  in Edge_type do
14:        if edge_type(node, connected_node)
15:          ==  $Edge_j$  then
16:             $Edge_j \leftarrow (\text{source\_node}, \text{node})$ 
17:            update  $X_j$ 
18:          end if
19:        end for
20:      end for
21:       $Edge \leftarrow Edge_j$ 
22:    end for
23:  end for
24: return  $X, L, E$ 

```

E4: A two-dimensional array of size $m4 * 2$. *m4* represents the number of edges of the WasDerivedFrom type. *E0*[0][0] and *E0*[0][1] represent the source node and target node of the first WasDerivedFrom edge, respectively.

(4) Integrate the graph label *L*, feature vector list *X*, edge arrays *E0*, *E1*, *E2*, *E3*, and *E4* obtained earlier to obtain the final IDS-HGAT dataset.

4.4. Redis storage

Considering that provenance collection, preprocessing, and intrusion detection are performed in parallel, and the processing speed of the two is generally different, there is a producer–consumer problem. An intermediate database is needed to decouple the two so that the provenance collection and preprocessing processes do not need to call the interface of the intrusion detection process. At the same time, the provenance collection and preprocessing process and the intrusion detection process can run asynchronously. For this reason, considering the high performance of the in-memory database Redis, this paper uses Redis as an intermediate database. It uses its stream type to construct a message queue to solve producers’ and consumers’ problems efficiently. In order to reduce the number of reads and writes to the Redis database when storing and obtaining the provenance graph, the data of each provenance graph obtained by preprocessing is constructed as a hash table, in which each key–value pair is a string type. Fig. 4 is an example of an abnormal provenance graph. The storage content of the provenance graph in the Redis database is shown in Table 2. Among them, the key *X* stores the feature vector of all nodes in the provenance graph. Each node’s feature vector is separated by the string “:”, and

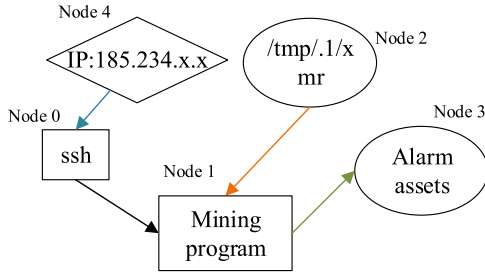


Fig. 4. Example of abnormal provenance graph.

Table 2

Example of storage content of provenance graph in Redis.

Key	Value
X	1 0 0 1 0: 1 0 0 1 0: 2 0 1 0 0: 2 0 0 0 0: 0 1 0 0 0
Y	1
E0_Source	\
E0_Target	\
E1_Source	2
E1_Target	1
E2_Source	1
E2_Target	3
E3_Source	0
E3_Target	1
E4_Source	\
E4_Target	\

its subscript corresponds to the number of the node, thereby uniquely labeling each node. For node 1, no edges take it as the source node. According to the node type of the target node, the type of the connected edge can be judged. It can be read from the table that the feature vector of node 1 is [1 0 0 1 0]. Secondly, the key Y stores the label of the provenance graph. A value of 1 indicates that the provenance graph is abnormal, and a value of 0 indicates that the provenance graph is normal. The other key-value pairs store the node pairs of each type of edge. For example, E2_Source and E2_Target store the source and target node pairs for “WasGeneratedBy” respectively. Since there is only one edge of this type between node 1 and node 3 in Fig. 4, the values of E0_Source and E0_Target are the strings “1” and “3”.

IDS-HGAT intrusion detection method will start multi-processes to read the data in Redis at the same time. In order to prevent multi-processes from repeatedly consuming the data stored in Redis, this module uses the consumer group mode of Redis. The consumer group alternately reads messages from the message queue to ensure that the messages in the message queue can be read without duplication and full coverage. In a specific implementation, use the xgroup function in the Redis-py library to create and manage consumer groups and use the library's xreadgroup function to read messages in groups.

4.5. Parallel IDS-HGAT intrusion detection

This section first trains the IDS-HGAT network to obtain an IDS-HGAT classifier. Then, the classifier is deployed to several intrusion detection processes, and the provenance data to be detected is read and classified from the Redis message queue in parallel.

4.5.1. IDS-HGAT implementation

This paper reforms the HAN network to get the IDS-HGAT intrusion detection method. The provenance graph collected by SPADE contains three types of nodes and five types of edges. The types of source and target nodes for each type of edge are fixed (for example, the source

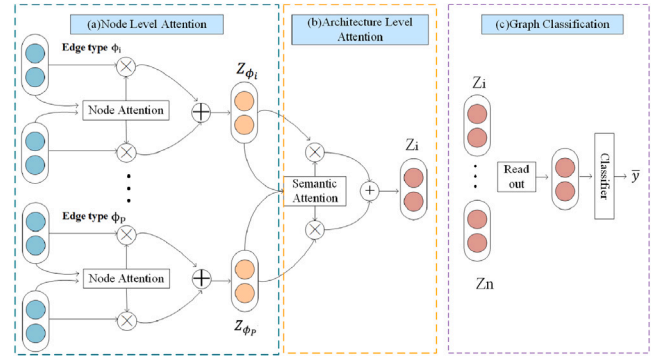


Fig. 5. IDS-HGAT intrusion detection.

node of an edge of the WasControlledBy type is a *process* type, and the target node is an *agent* type). The neighbors that each node is connected to by different types of edges have different effects on that node. Therefore, unlike HAN, IDS-HGAT does not manually specify the meta-path, but rather distinguishes the neighbors of each node in the provenance graph based on five different provenance edge types. Fig. 5 shows the IDS-HGAT intrusion detection method.

There is a node set $N_i = \{j, k, \dots\}$, ($i \neq j \neq k$) connected to node i in the provenance graph via the provenance edge, and the node set N_i is all the neighbors of node i . We learn the attention vector group Z_{ϕ_i} , ($i = 0, \dots, P$) of the connected different neighbors to the node i based on different edge types ϕ_i , ($i = 0, \dots, P$), the vector group represents the influence on node i by neighbors of different types of edge connections. The process of obtaining the attention vector group is called the node-level attention mechanism. The attention vector Z of the node i is then obtained by aggregating the learned attention vector Z_{ϕ_i} and assigning different weights to different edges. Because the goal of IDS-HGAT is to classify the provenance graph, we add a graph pooling layer after the attention layer of the architecture level to integrate the attention vectors of all nodes as the overall provenance graph attention vector O . After that, the task of graph classification can be performed based on the attention vector of the provenance graph.

We consider the neighbors of five edge types of each node and use the node-level attention mechanism to learn the importance of various types of neighbors. As shown in Eq. (1).

$$e_{ij}^{\phi_i} = \text{attention}_{\text{node}}(h_i, h_j; \phi_i) \quad (1)$$

Here $\text{attention}_{\text{node}}$ represents a deep neural network when performing node-level attention. The neural network is shared by a given node i and its neighbors of the same edge type ϕ_i connected to it.

To better compare the weight coefficient between different node pairs, the method uses the softmax function to normalize it to express.

$$\alpha_{ij}^{\phi_i} = \text{softmax}(e_{ij}^{\phi_i}) \quad (2)$$

Then, by aggregating the features of neighbors and the corresponding weight coefficients learned earlier, we can obtain the embedding of node i based on edge type ϕ_i .

$$z_i^{\phi_i} = \sigma \left(\sum_{j \in N_i^{\phi_i}} \alpha_{ij}^{\phi_i} \cdot h_j \right) \quad (3)$$

where $z_i^{\phi_i}$ is the embedding of node i for edge type ϕ_i . We use a multi-head attention mechanism to stabilize the learning process of the node-level attention mechanism.

$$z_i^{\phi_i} = \parallel_{k=1}^K \sigma \left(\sum_{j \in N_i^{\phi_i}} \alpha_{ij}^{\phi_i} \cdot h_j \right) \quad (4)$$

Given the edge type set $\{\phi_0, \phi_1, \dots, \phi_P\}$, we can get the corresponding node embeddings, that is, $\{Z_{\phi_0}, Z_{\phi_1}, \dots, Z_{\phi_P}\}$.

To make the learned node embeddings more comprehensive, we need to fuse multiple semantic information from different edge types. We average the importance of all node embeddings with semantic information, which can be considered the importance of each edge type. The importance of each edge type is expressed as:

$$w_{\phi_i} = \frac{1}{|V|} \sum_{i \in V} q^T \cdot \tanh(W \cdot z_i^{\phi_i} + b) \quad (5)$$

Where W is the weight matrix, b is the bias vector, and q is the attention vector at the architecture level. Note that all semantic-based embeddings share all the above parameters for meaningful comparison. After obtaining the importance of each edge type, we normalize it with the softmax function.

$$\beta_{\phi_i} = \text{softmax}(w_{\phi_i}) \quad (6)$$

It can be interpreted as the importance of a particular edge type. We use the learned weights as coefficients and fuse the corresponding node-level embedding to obtain the final embedding Z , as shown below:

$$Z = \sum_{i=1}^P \beta_{\phi_i} \cdot Z_{\phi_i} \quad (7)$$

By using three different global graph pooling layers, the embedding of all nodes in the graph is integrated into the attention vector of the whole graph.

- Cumulative pooling

$$O = \sum_{n=1}^N Z_n \quad (8)$$

- Average pooling

$$O = \frac{1}{N} \sum_{n=1}^N Z_n \quad (9)$$

- Maximum pooling

$$O = \max_{n=1}^N Z_n \quad (10)$$

Where O represents the provenance graph's embedding, N represents the set of nodes in the provenance graph, and Z_n represents the embedding of node n .

Based on the learned provenance graph embedding, a layer of linear function is used to obtain the prediction of the provenance graph label.

$$\bar{y} = \text{Linear}(O) \quad (11)$$

Where $\bar{y}$ is the predicted label, and Linear is a linear regression function commonly used in binary classification tasks.

The implementation of this model is shown in Algorithm 3.

4.5.2. Parallel intrusion detection

In order to improve detection speed, this paper starts multiple IDS-HGAT intrusion detection processes. It retrieves provenance data from the Redis database for intrusion detection in parallel to fully exploit the advantages of modern computer multi-core. However, parallel intrusion detection will also cause related problems. First of all, multiple processes read data from the database simultaneously, slowing down the reading speed, repeating reading, or missing reading. As a result, we create a consumer group for Redis' message queue. The solution sets the provenance intrusion detection process as the consumer so that multiple processes can read data efficiently and ensure each piece of data can be read without repetition. Secondly, there is no fixed standard for the number of intrusion detection processes. It is related to the number of processes of the producer, so it needs to be obtained through testing. As shown in Fig. 2, several consumer groups are created in

Algorithm 3 IDS-HGAT model

Input: $G = (V, E)$; $h_i, i \in V$; $\{\phi_1, \phi_2, \dots, \phi_P\}$;
 K
 // $G = (V, E)$ represents provenance graph
 // h_i represents node feature vector
 // ϕ_i represents edge type set
 // K represents attention head number
Output: O // Provenance graph embedding
 $\bar{y}$ // Classification prediction label

```

1: for  $\phi_i \in \{\phi_0, \phi_1, \dots, \phi_P\}$  do
2:   for  $k = 1 \dots K$  do
3:     for  $i \in V$  do
4:       Find neighbors  $N_i$  based edge type  $\phi$ 
5:       for  $j \in N_i^\phi$  do
6:          $\alpha_{ij}^\phi = \text{softmax}(\text{att}_{\text{node}}(h_i, h_j; \phi))$ 
7:       end for
8:        $z_i^\phi = \sigma \left( \sum_{j \in N_i^\phi} \alpha_{ij}^\phi \cdot h_j \right)$ 
9:     end for
10:     $z_i^\phi = ||_{k=1}^K z_i^\phi$ 
11:  end for
12:   $\sum_{i=1}^P \beta_{\phi_i} \cdot Z_{\phi_i} \rightarrow Z$ 
13: end for
14:  $\sum_{n=1}^N Z_n \rightarrow O$ 
15: Predict label  $\text{Linear}(O) \rightarrow \bar{y}$ 
16: return  $O, \bar{y}$ 

```

Algorithm 4 Redis-Based Parallel Intrusion Detection

Input: *ProvenanceStream*
Output: *IntrusionDetectionResults*

```

1: Initialize Redis client connection
2: Create a consumer group for the stream ProvenanceStream
3: for each process in the pool do
4:   Assign Consumer_i to the process
5:   while True do
6:     Read a message m_i from the Redis stream
7:     if m_i contains provenance graph then
8:       Extract provenance_graph and label from m_i
9:       Run provenance_graph through the IDS-HGAT
        model
10:      Obtain the classification result from IDS-HGAT
11:      if result == 'anomaly' then
12:        Flag and store the anomaly result
13:      end if
14:    end if
15:    Acknowledge message m_i to mark as processed
16:    if no new message for a period then
17:      Exit the process // No more messages to process
18:    end if
19:  end while
20: end for
21: Wait for all processes to finish
22: Return intrusion detection results

```

Redis. Consumers in the same consumer group can take out the provenance data in the Redis message queue in parallel without error. Then IDS-HGAT creates a corresponding intrusion detection process. Each process corresponds to a consumer in the consumer group and performs intrusion detection in parallel. Algorithm 4 shows pseudo-code for intrusion detection using Redis parallel processing.

Table 3
Host configuration.

Host	Operating system	CPU	RAM	Solid state drive	Hard disk
Host1	Windows10	Intel Core i7-7700 CPU @ 3.60GHz	32GB	Samsung SSD 860 PRO 256 GB	\
Host2	Ubuntu 16.04 LTS	Intel(R) Xeon(R) CPU E5-2630 v4 @ 2.20GHz	94 GB	\	1T

5. Experimental evaluation

In this section, we first describe the experimental environment and datasets and then evaluate the effectiveness of our method from the following four aspects:

Q1. How to set and optimize parameters to get better detection results? (Section 5.2)

Q2. How effective is IDS-HGAT compared to existing state-of-the-art methods? (Sections 5.3 and 5.5)

Q3. Does IDS-HGAT alleviate the space overhead after preprocessing? (Section 5.4)

Q4. Does Redis-based parallel intrusion detection improve detection speed? (Section 5.5.1)

We take the Samba dataset as an example, obtain the optimal parameters, high precision, and low false alarm rate. The necessity of tuning parameters is explained (Q1). We also compare the space overhead after dataset preprocessing, illustrating the effectiveness of this process (Q3). We conducted experiments on five datasets and compared them with UNICORN and GAT. The results show that it is superior in both accuracy and time, proving the effectiveness of IDS-HGAT (Q2). Finally, we use Redis-based parallel intrusion detection, showing that it can indeed speed up detection (Q4).

5.1. Experimental setup

Test Environment. The experiments are run on two hosts, one is a Windows system and the other is an Ubuntu system. The Windows system is used for IDS-HGAT intrusion detection, and the Ubuntu system is used for provenance preprocessing and building the Redis database. The host configuration is shown in Table 3.

Evaluation Criteria. We convert provenance-based intrusion detection into the classification task of the provenance graph. According to whether there is an intrusion in the provenance, the provenance graph can be classified into two categories: abnormal and normal. We regard the abnormal provenance graph as positive and the normal provenance graph as negative.

We use the following precision and false alarm rate to evaluate the method.

(1) Precision

$$Precision = TP / (TP + FP) \quad (12)$$

(2) False alarm rate

$$False\ alarm\ rate = FN / (TP + FN) \quad (13)$$

TP (True Positive) indicates the number of times that the abnormal provenance graph is correctly predicted as abnormal. FN (False Negative) indicates the number of the abnormal provenance graph is incorrectly predicted as normal. FP (False Positive) indicates that the normal provenance graph is incorrectly predicted as abnormal. TN (True Negative) indicates the number of the normal provenance graph is correctly predicted as normal.

Experimental Dataset. We use six datasets, and the details are shown in Table 4. The first four datasets were collected using the SPADE provenance collection system, and are available online at <http://yulaixie.cn/>. The SC1 dataset [6], which was generated using CamFlow (v0.5.0) to capture whole-system provenance of system activity, including background activity during both benign and attack scenarios. Due to the large size of the SC1 dataset obtained, we randomly selected a part of it as a substitute for verification, with a total size of 8.26 GB. The DARPA-OpTC dataset [46] was collected in a technology transition

pilot study funded by Boston Fusion's Enterprise, which targeted the Advanced Persistent Threat (APT) scenarios of its CASES project.

(1) Samba application collection

The abnormal files in the Samba dataset were collected by simulating an attack by exploiting vulnerability in the Samba. We simulated the normal behavior for 100 times and extracted 95,310 normal nodes, including 2851 processes, 92,394 files and 65 agents. Then, 50 attacks were simulated, and a total of 17,411 nodes were extracted, of which 3657 were tested processes, of which 50 were abnormal processes, and 3607 were normal processes.

(2) Tomcat application provenance collection

The abnormal files in the Tomcat dataset were collected by simulating an attack by exploiting vulnerability in the Tomcat. We simulated the normal behavior for 100 times and extracted a total of 19,336 nodes, including 2013 normal process features. Then 50 attacks were simulated, and a total of 5106 nodes were extracted, including 1559 test processes, including 50 abnormal process characteristics, and 1509 normal process characteristics.

(3) Webmin application provenance collection

The abnormal files in the Webmin dataset were collected by simulating an attack by exploiting vulnerability in the Webmin. We simulated the normal behavior for 100 times and extracted a total of 108,832 nodes, including 2428 normal process features. Then 50 attacks were simulated, and a total of 13,102 nodes were extracted, including 4535 test processes, including 50 abnormal process characteristics, and 4485 normal process characteristics.

(4) WordPress application provenance collection

The abnormal files in the WordPress dataset were collected by simulating an attack by exploiting vulnerability in the WordPress. We simulated the normal behavior for 100 times and extracted a total of 14,186 nodes, including 2131 normal process features. Then 50 attacks were simulated, and a total of 11,373 nodes were extracted, including 2434 test processes, including 50 abnormal process characteristics, and 2384 normal process characteristics.

(5) SC1 provenance collection

This dataset was collected using CamFlow in a controlled laboratory environment, which simulates APT supply-chain attack datasets and captures the entire system provenance, including normal and abnormal behavior.

(6) DARPA-OpTC provenance collection

The DARPA-OpTC dataset is system data collected by the host, including benign data over seven days and a mixture of APT attacks and benign activities over three days. APT attacks are carried out by a red team using known CVEs on a subset of hosts, simulating real attack scenarios. The size of this dataset is approximately tens of terabytes, which can help us better simulate operations in real environments. In the test set, data from three hosts were used on different dates (the 23rd, 24th, and 25th), with attack activity occurring only during specific periods and normal behavior throughout the rest of the time.

5.2. Hyperparameter test

In this section, we describe the analysis and tuning with different parameters on the samba dataset.

(1) *Graph pooling layer.* We tried 100 hyperparameter combinations for each graph pooling method, then selected the best hyperparameter combination from each method, and finally compared the effects of the three pooling methods.

The convergence of loss functions for three pooling methods is illustrated in Fig. 6. We use the mean pooling, add pooling, and max pooling

Table 4
Dataset information.

Dataset	Normal provenance			Abnormal provenance		
	Files number	Nodes number	Edges number	Files number	Nodes number	Edges number
Samba	100	95,310	294,960	50	17,411	78,574
Tomcat	100	19,336	119,853	50	5106	46,372
Webmin	100	108,832	320,104	50	13,102	167,041
WordPress	100	14,186	119,557	50	11,373	69,994
SC1	125	2883,757	9779,199	25	1578,488	4780,264
DARPA-OpTC	13	3907,089	28,493,183	6	925,165	594,769

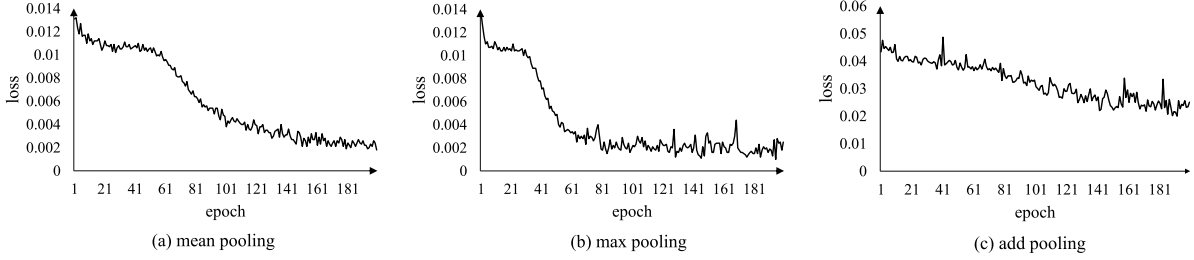


Fig. 6. Graph pooling layer test results.

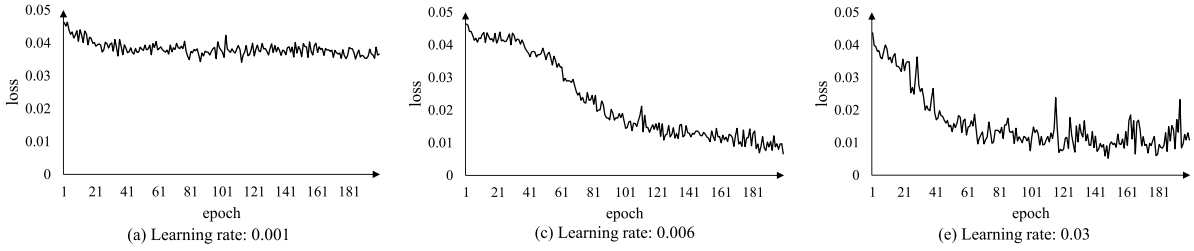


Fig. 7. The test results of learning rate in Samba dataset.

Table 5
Intrusion detection results when different pooling methods are selected for the Samba dataset.

Pooling method	Results	
	Precision	False alarm rate
mean pooling	1	0
max pooling	0.75	0
add pooling	0.6939	0

Table 6
Intrusion detection results at different learning rates in Samba dataset.

Learning rate	Indicators	
	Precision	False alarm rate
0.001	0.5	0
0.006	0.973	0
0.03	0.6667	0

in the PyTorch geometry_library to realize the average, accumulation, and maximum pooling. Obviously, the loss function converges best when the graph pooling layer is mean pooling. The test results are shown in Table 5. When the graph pooling layers are set to max pooling and add pooling, the precision is very low, and the false alarm rate is high. Compared with max pooling, mean pooling and add pooling can comprehensively consider more graph information, so the intrusion detection results are more stable. Add pooling is more difficult to converge than mean pooling and requires more training epochs. We finally chose to mean pooling as the pooling layer of IDS-HGAT.

(2) *Learning rate.* Learning rate is one of the most critical hyperparameters that determines the step size of the method moving towards the minimum value of the loss function. When setting the learning

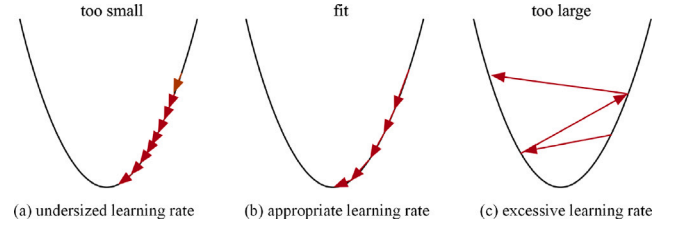


Fig. 8. Convergence under different learning rates.

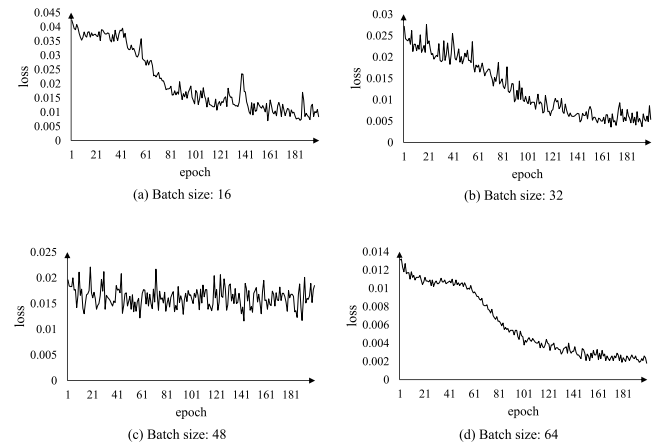


Fig. 9. The test results of batch size in Samba dataset.

Table 7
Intrusion detection results for Samba dataset with different batch sizes.

Batch size	Indicators	
	Precision	False alarm rate
16	0.973	0
32	0.9231	0
48	0.5	0
64	0.9231	0

rate, there is a trade-off between convergence speed and overshooting speed. Although the gradient of the loss function usually determines the direction of descent, the learning rate determines the length of the step taken in that direction. A high learning rate will make the model skip the minimum value and fail to converge, but a low learning rate will take too long to converge or fall into the undesired local minimum value [47]. Fig. 8 shows the convergence process of the network at different learning rates.

We fixed the number of attention heads at 8, then set the batch size at 16 and the number of training epochs at 200 to test the effect of the commonly used learning rates on the model. The test results are shown in Fig. 7. Too high and too low learning rates will cause the loss function to fail to converge. Because the learning rate is too high, the model may miss the trough of the gradient function in the process of gradient descent, constantly wandering on both sides of the trough, failing convergence. If the learning rate is too low, it is easy to fall into a local trough in a gradient descent, leading to converging failure. We measured 0.006 as the optimal learning rate, and the corresponding curve nearly converged around epoch 200. The test results of each learning rate model are shown in Table 6. The intrusion detection results are best when the learning rate is 0.006, which is in line with the result of the loss function graph.

(3) *Batch size.* The batch size determines the amount of data for each model of training and is usually smaller than the overall data amount. One reason for training batch sizes of data rather than all data is to save memory. The amount of data used for training is generally large, and loading it all into memory at once is often impractical. In addition, using a smaller batch size can lead to faster model convergence because smaller batch sizes mean faster parameter updates. Therefore, it is necessary to choose the appropriate batch size.

We fixed the attention head at 8 and the learning rate at 0.006, then set the number of training epochs at 200 to test the influence of the commonly used batch size on the model. The loss test results are shown in Fig. 9. Table 7 shows the detection effect under different batch sizes. Obviously, the intrusion detection results are best when the batch size is 16.

(4) *Number of attention heads.* In order to stabilize the learning process of self-attention, multi-head attention is used in the IDS-HGAT intrusion detection method. A multi-head attention mechanism can improve the accuracy of model learning and make the model more stable. Too few attention heads may lead to unstable learning, but too many attention heads will lead to redundancy between heads, slowing down the training process and other problems. Therefore, it is necessary to select the appropriate number of attention heads through experiments.

We fixed the batch size at 16, the learning rate at 0.006, and the training rounds at 200 to test the influence of the commonly used attention heads. We used a common set of attention heads, respectively 1, 2, 4, and 8. The loss test results are shown in Fig. 10. The best results were obtained when the attention heads were 8. The loss function fails to converge when the attention heads are 1, 2, and 4. As shown in Table 8, the intrusion detection result is the best when the attention head is set to 8, which is in line with the result of the loss function graph. In summary, the combination of hyperparameters we chose on the Samba dataset is: the number of attention heads is set to 8, the learning rate is 0.006, and the batch size is 16.

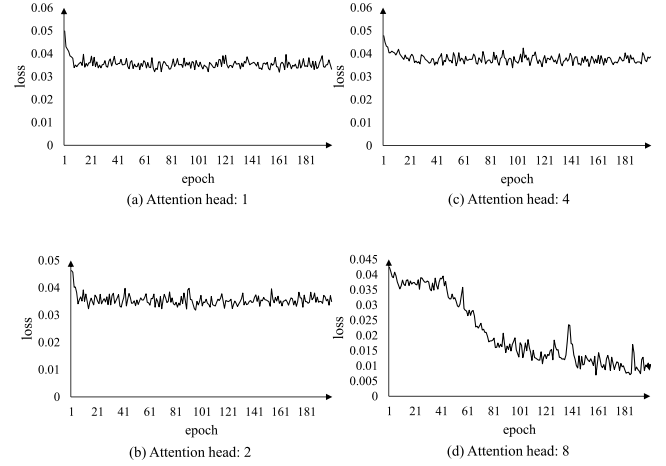


Fig. 10. The test of attention heads in Samba dataset.

Table 8
Intrusion detection results with different attention heads in Samba dataset.

Attention heads	Indicators	
	Precision	False alarm rate
1	0.6531	0
2	0.5455	0
4	0.2759	0
8	0.973	0

We have performed the above hyperparameter tuning on all datasets. The optimal hyperparameter combination and intrusion detection results are shown in Table 9. IDS-HGAT can obtain high detection precision and low false alarm rate.

5.3. Compare the detection performance of different systems

This section compares IDS-HGAT with the intrusion detection method based on GAT, and state-of-the-art intrusion detection methods (UNICORN [6], THREATTRACE [19], MAGIC [5], KAIROS [48]). We use the experimental results obtained under the optimal parameters in Section 5.2 for comparison.

(1) Intrusion detection method based on GAT

We implement an intrusion detection model based on GAT and compares it with IDS-HGAT. The GAT network defines a single layer of self-attention to learn the influence of neighbor nodes but does not consider the situation of heterogeneous graphs. It is a 3-layer feed-forward network. The zeroth layer of the network is the characteristic of the input node. Since the node has five characteristics, the number of nodes in this layer is 5. The first to third layers of the network use 16 self-attention nodes, and finally, a sigmoid layer is used, with an output dimension of two. The binary classification task is performed to determine the category to which the provenance graph belongs.

(2) State-of-the-art intrusion detection methods

UNICORN: UNICORN periodically constructs a histogram stream from provenance data, updating it to maintain relevant context. It uses similarity-preserving hashing to convert histograms into sketches, then clusters them using the K-medoids method to represent system meta-states. These sketches and statistics create a model of system evolution, incorporating training provenance graphs to detect intrusions in test sets.

THREATTRACE: THREATTRACE is a system designed for threat detection in streaming graph data. It employs a multi-model framework and probability-based methods to reduce false positives and false negatives. By dynamically maintaining a subgraph in memory and leveraging feature extraction techniques, THREATTRACE can efficiently detect

Table 9
Intrusion detection results under optimal hyperparameters of each dataset.

Dataset	Hyperparameters			Indicators	
	Attention heads	Learning rate	Batch size	Precision	False alarm rate
Samba	8	0.006	16	0.973	0
Tomcat	8	0.001	16	1	0.0714
Webmin	8	0.001	48	1	0
WordPress	4	0.1	48	1	0.0526
SC1	4	0.003	1	1	0

Table 10
Hyperparameters used in GAT model.

Dataset	Hyperparameters		
	Hidden layer	Batch size	Learning rate
Samba	16	16	0.013
Tomcat	16	16	0.003
Webmin	16	16	0.03
WordPress	16	32	0.013
SC1	16	32	0.013

Table 11
The precision and false alarm rate of the three intrusion detection methods.

Dataset	Precision			False alarm rate		
	GAT	UNICORN	IDS-HGAT	GAT	UNICORN	IDS-HGAT
Samba	0.98	0.98	0.973	0	0	0
Tomcat	0.9615	0.6	1	0	1	0.0714
Webmin	0.9423	1	1	0.1111	0.96	0
WordPress	0.8846	0	1	0.1905	0	0.0526

Table 12
Comparison of IDS-HGAT and state-of-the-art detection methods on SC1 datasets.

Dataset	Methods	Precision	Recall	Accuracy	F1-score	FPR
SC1	UNICORN	0.85	0.96	0.90	0.90	0.155
	THREATTRACE	0.93	0.98	0.953	0.95	0.074
	MAGIC	0.98	0.96	0.909	0.97	0.020
	IDS-HGAT	1	1	1	1	0

anomalies in real-time while learning from the entire graph during training.

MAGIC: MAGIC constructs a provenance graph from audit logs and employs a graph representation module to generate embeddings, incorporating graph features and structural information in a self-supervised manner. The model is based on graph masked auto-encoders and is supervised for masked feature reconstruction and sample-based structure reconstruction. Finally, an unsupervised outlier detection method is used to analyze the embeddings and achieve the detection outcome.

The IDS-HGAT intrusion detection system proposed is compared with the UNICORN intrusion detection system and the intrusion detection system based on GAT, respectively. The STD value of UNICORN is set to 8.3, which has the best average detection effect. The hyperparameter combination used by GAT is shown in Table 10.

The precision and false alarm rate of the three methods on the dataset collected by the SPADE system are shown in Table 11. Compared with UNICORN and GAT, IDS-HGAT has higher precision and lower false alarm rate. It can be seen that GAT based on graph neural networks and IDS-HGAT are significantly better than UNICORN. UNICORN's effect on the Tomcat, Webmin, and WordPress datasets is very unsatisfactory. UNICORN is a streaming calculation method that uses sliding windows to extract provenance graph features. The SPADE generates batch provenance data by period. UNICORN cannot fully detect the provenance graph, so it is easy to extract parts of the features of abnormal instances and classify them as normal instances. Furthermore, since GAT network does not divide all nodes in the provenance graph into multiple types, it simply learns the attention vector of the entire provenance graph. In contrast, IDS-HGAT adopts a hierarchical attention mechanism, which considers different types of nodes and

edges and better learns the overall structure of the provenance graph, so IDS-HGAT can better extract the information from the provenance graph.

As can be seen from Table 11, the precision of IDS-HGAT intrusion detection method reaches 100% on most datasets, while the false alarm rate does not exceed 7.2%. The detection performance is significantly better than UNICORN and GAT.

In addition, Table 12 presents the comparison results between IDS-HGAT and several state-of-the-art intrusion detection methods on the SC1 dataset. Compared with these graph-level granularity methods, IDS-HGAT achieved the best results, indicating its effectiveness in detecting system anomalies.

To further evaluate the performance of IDS-HGAT, we additionally report the ROC (Receiver Operating Characteristic) curves and corresponding AUC (Area Under the Curve) scores on four datasets (Samba, Tomcat, WordPress, and Webmin). As shown in Fig. 11, IDS-HGAT achieves high AUC scores on all four datasets, which indicates its strong discriminative capability between normal and abnormal provenance graphs. Hierarchical attention mechanism effectively captures both the semantic and structural differences across various types of nodes and edges. The model's ability to focus on important substructures, combined with graph-level pooling, enhances the overall graph representation.

(3) Comparison with the real-time detection method

KAIROS: KAIROS is an encoder-decoder architecture based on graph neural networks that learns temporal evolutionary structural changes in source graphs to quantify the degree of anomaly in each system event. KAIROS then uses this fine-grained information to reconstruct the attack footprint, generating a compact summary graph that accurately depicts malicious activity in the system's audit log stream.

The parameters follow the same acquisition method as our other datasets (the optimal parameter combination are: Learning rate: 0.01, Batch size: 2, attention heads: 4). Experimental results are shown in Table 13, IDS-HGAT achieves higher accuracy (0.90 vs. 0.84), demonstrating its effectiveness in distinguishing benign from malicious activities. KAIROS achieves a higher recall, possibly due to its real-time detection mechanism, allowing it to track evolving attack patterns continuously. In contrast, IDS-HGAT simulates a dynamic environment by analyzing time-segmented provenance graphs, which may result in missing some attacks.

In order to better reflect the detection capability of IDS-HGAT in dynamic application environments, we split the three days of provenance data after the attack by the time of normal and abnormal activity. Each day of the APT attack cycle is further broken down into hour intervals, with attacks and benign activity occurring at different times. For example, on the third day of APT attacks, the attacks occurred between 06:00 and 12:00, another attack occurred between 18:00 and 23:59, and the rest of the time were benign activities. The data within each time interval is similar to a subgraph. This dataset is based on temporal continuity and divides the data in chronological order, allowing the detection process to simulate the effect of dynamic detection.

Case study. We compare the attack subgraph obtained under the abnormal label with the Ground Truth provided by DARPA to verify the efficacy of IDS-HGAT. Simultaneously, we can obtain the attack scenario in the Ground Truth, as shown in Fig. 12. The attacker

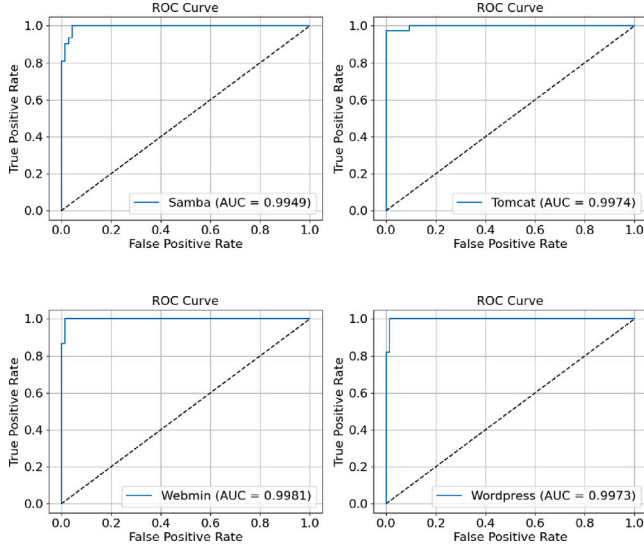


Fig. 11. ROC curve (AUC) results for four datasets.

Table 13

Comparison of IDS-HGAT and the real-time detection method on DARPA-OpTC dataset.

Dataset	Methods	Accuracy	Precision	Recall
DARPA-OpTC	KAIIROS	0.84	1	0.99
	IDS-HGAT	0.90	1	0.75

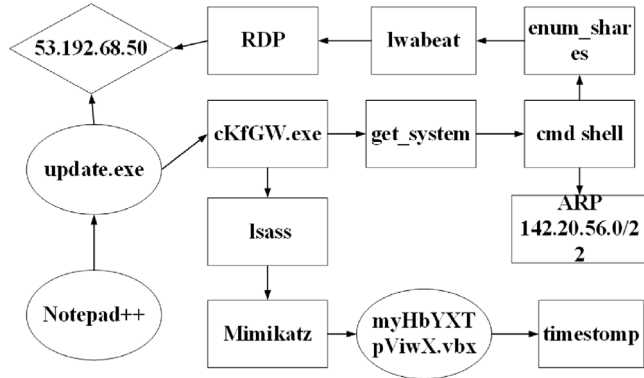


Fig. 12. A concise graph that describes attack activity in DARPA's OpTC dataset in day 3.

implants Trojan *update.exe* through Notepad++ malicious upgrade, reverse connects to the C2 server (53.192.68.50:8080) to obtain system permissions, and then uses Meterpreter to perform Intranet ARP scanning and share enumeration. After locating the target, the *lsass* process is injected to steal the plaintext credentials and spread horizontally. Using registry persistence scripts and Timestomp to tamper with file time and cover up their movements. Ultimately, malicious processes are migrated to *lwabeat* disguised as legitimate services, and RDP backdoor users are created to maintain covert control.

5.4. Space overhead

We compare the space overhead of the five datasets after IDS-HGAT preprocessing with the original datasets, and the results are shown in Fig. 13. It obviously shows that the space occupied by the preprocessed data is much less than the original data. Specifically, after preprocessing, the compression rates of the five data sets are 12.7%, 12.7%, 13.1%, 12.8% and 6.5%, respectively. It shows that the

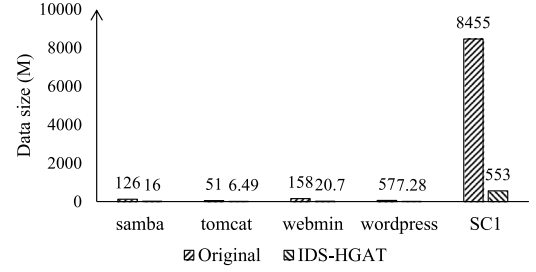


Fig. 13. Space overhead for original dataset and processed dataset.

Table 14

Comparison of data compression on TRACE dataset (CR represents the Compression Rate).

Methods	Original data	After reduction	CR
SHADEWATCHER	19.2G	983MB	95%
FLASH	19.2G	408MB	98%
IDS-HGAT	19.2G	170MB	99%

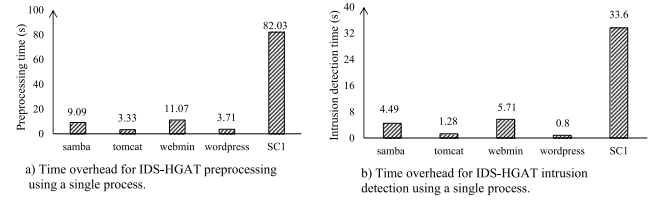


Fig. 14. Time overhead for IDS-HGAT preprocessing and intrusion detection using a single process.

proportion of nodes and edges irrelevant to intrusion detection is close to 13% on the SPADE datasets and close to 6.5% on the SC1 dataset. The result demonstrates the effectiveness of IDS-HGAT preprocessing.

To further validate the effectiveness of our preprocessing, we implement similar noise reduction as FLASH [22] and SHADEWATCHER [16]. FLASH as node-level detection cuts noise through selective graph traversal and semantic abstraction. FLASH filters low priority events (such as temporary file deletion operations), merges nodes with the same execution attributes (such as processes with the same path but different pids), and ignores user-specific differences (such as user ids in the path). At the same time, Word2Vec is used to encode semantic attributes (such as process name and file path), and time series information is combined to generate compact node embeddings. SHADEWATCHER detects the interaction memory between entities. By merging repetitive causal events (such as reading and writing the same file multiple times and only retaining the first record), Shade-Watcher removes isolated temporary files and trust library accesses, splits the provenance graph into behavioral subgraphs, and only keeps the key interactions to construct a bipartite graph. Our preprocessing, on the other hand, eliminates irrelevant nodes (e.g., unnamed temporary files, system libraries) while explicitly reconnecting their parents and children via edge links to preserve the topology of the provenance graph. Considering the main noise reduction strategies, we perform a compression comparison on the public DARPA TRACE dataset [49]. The results, shown in Table 14, show high compression ratios achieved by all three processing methods (98% for FLASH, 95% for SHADEWATCHER, and 99% for IDS-HGAT). The results show that our method achieves comparable or better size reduction without sacrificing structural consistency.

5.5. Time overhead

We first test the time cost of IDS-HGAT preprocessing and IDS-HGAT intrusion detection on five datasets, respectively, and then test the time

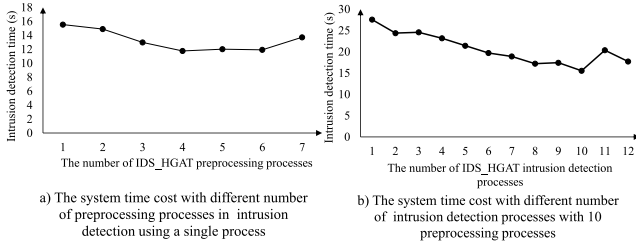


Fig. 15. Time overhead of parallel intrusion detection system based on Redis.

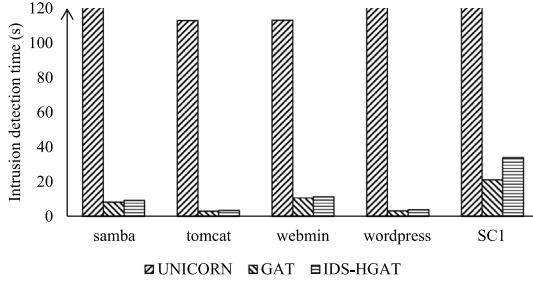


Fig. 16. Detection time of different intrusion detection methods.

cost of the parallel intrusion detection system on SPADE datasets using Redis. Finally, we compared the time costs of three intrusion detection systems.

5.5.1. Time overhead of IDS-HGAT

The time cost of preprocessing and intrusion detection using a single process is shown in Fig. 14. As can be seen from Fig. 13, the size of the Tomcat dataset and the WordPress dataset is only 51M and 57M, respectively, so the time spent on preprocessing and detection is relatively low, as shown in Fig. 14. However, the size of SC1 dataset is 8455M, so more nodes and edges need to be processed when generating data objects, and more data are read during detection, so the time cost is high. In order to improve the detection efficiency of IDS-HGAT, we use Redis to speed up the detection process. Fig. 15 shows the time cost of a Redis-based parallel intrusion detection system on the SPADE dataset. Subfigure (a) shows the system time cost of different preprocessing processes in intrusion detection using a single process. The results show that the intrusion detection time first decreases and then increases with the number of preprocessing processes. Thus, the optimal number of preprocessing processes is 10, which is exactly the preprocessing host's number of CPU cores. Subfigure (b) shows the time cost of different intrusion detection processes when the number of preprocessing processes is fixed at 10. It can be seen that when the number of intrusion detection processes is 4, the time overhead is 11.76s, which is 70.21% lower than the time cost of the IDS-HGAT intrusion detection system using a single process.

5.5.2. Comparison of detection time of different intrusion detection systems

We tested the detection time of three intrusion detection systems on five datasets. We execute multiple times and then take the average value, as shown in Fig. 16. The detection time of GAT and IDS-HGAT is much lower than that of UNICORN. Because UNICORN is a real-time update intrusion detection system, it needs to learn while testing. Moreover, UNICORN uses a sliding window to process a single node at a time, and the algorithm complexity is considerable. Once the GAT and IDS-HGAT models are trained, the testing process takes very little time. The detection time of UNICORN is much larger than the ordinate value in the figure (i.e., 120 s) on the three datasets of Samba, WordPress, and SC1. We have not marked it for clarity. The detection time on these three datasets are 386 s, 545.9 s, and 3448 s, respectively.

6. Discussion and limitations

IDS-HGAT demonstrates promising performance in detecting APT attacks through provenance graph analysis, achieving high precision and low false alarm rates across multiple datasets. However, several limitations and open challenges remain, which we discuss below.

First, attackers complete the attack to deceive the target system by constructing a specific sample without changing the target model [50]. The constructed samples are often referred to as “adversarial samples”. For example, an attacker can modify the non-critical features of an abnormal sample, causing the model to judge it as a benign sample, thereby bypassing detection. Therefore, we can construct adversarial examples in advance in the training phase to train the model. As the number of training increases, the accuracy of the model will increase, and the robustness of the model will also increase. The possibility of being attacked by adversarial examples becomes lower. Second, we can try to check the adversarial examples after building the model, that is, we can predict the probability that the input sample is an adversarial example [51]. But in fact, due to the complex structure of the provenance graph, it is difficult for an attacker to ensure that it can evade detection without affecting the attack effect. To ensure stability, we can regularly update the model to keep it in the new state and reduce the possibility of false positives. This can be explained as the more accurate the generated model is when the training dataset contains more behavioral patterns of attacks.

Attackers also often perform variable perturbations on the training data to change the model behavior and reduce the performance of the model (i.e., poisoning attacks). For example, attackers often like to add edges or connect two to influence the model. In general, two nodes that would not be connected are connected, and the edges between them may be adversarial. A common method is to compare the similarity of two end nodes to determine whether the edge is abnormal. The purified data is then used for model training [52–54].

We acknowledge that more advanced attack techniques can be used to evade attacks, but this is a problem encountered by many detection systems, not just this paper. This work does not perform formal evaluation or simulation of adversarial attack scenarios, nor does it offer fully implemented countermeasures. The analysis provided is conceptual and reflects common considerations in provenance-based detection systems. Addressing robustness to sophisticated adversarial threats remains an important direction for future research. In this paper, our primary focus is on effective detection of abnormal behavior patterns rather than developing comprehensive defenses against adversarial manipulation.

7. Conclusion

We present IDS-HGAT, a novel provenance-based intrusion detection system based on a heterogeneous graph attention network. IDS-HGAT simultaneously considers the semantic information of nodes in the provenance graph and the structural information of the provenance graph, which improves the accuracy of intrusion detection. In addition, it utilizes multi-processes to read the provenance graph from Redis in parallel, which improves the efficiency of intrusion detection. Our experimental evaluation shows that IDS-HGAT can detect anomalies with high precision and a low false alarm rate. Moreover, parallel detection reduces time overhead. In the future, we will explore more advanced graph neural network architectures to improve the performance of IDS-HGAT when dealing with more complex network attack patterns.

CRedit authorship contribution statement

Lin Wu: Writing – review & editing, Writing – original draft, Visualization, Validation, Methodology, Data curation. **Yu-Lai Xie:** Writing – review & editing, Project administration, Funding acquisition. **Shi-Xun Zhao:** Methodology, Investigation, Conceptualization. **Pan Zhou:** Formal analysis. **Dan Feng:** Supervision, Project administration. **Avani Wildani:** Writing – review & editing, Supervision. **Ya-Feng Wu:** Formal analysis.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported in part by the National Key R&D Program of China under Grant No. 2022YFB4501300, the Shenzhen Science and Technology Program, China (No. JCYJ20230807143706014) and the National Science Foundation of China under Grant No. 61972449.

Data availability

Data will be made available on request.

References

- [1] cnBeta, 2021, <https://www.cnbeta.com/articles/tech/1130609.htm>.
- [2] Hacker News, 2021, <https://hackernews.cc/archives/37186>.
- [3] Q. Cao, Y. Li, Z. Wu, Y. Miao, J. Liu, Privacy-preserving conjunctive keyword search on encrypted data with enhanced fine-grained access control, in: World Wide Web, 2019.
- [4] J. Ning, X. Huang, W. Susilo, K. Liang, Y. Zhang, Dual access control for cloud-based data storage and sharing, in: IEEE Transactions on Dependable and Secure Computing, 2020, p. 99, 1-1.
- [5] Zian Jia, Yun Xiong, Yuhong Nan, Yao Zhang, Jinjing Zhao, Mi Wen, Magic: Detecting advanced persistent threats via masked graph representation learning, in: Usenix Security Symposium (USENIX Security '23 Winter), 2023.
- [6] X. Han, T. Pasquier, A. Bates, J. Mickens, M. Seltzer, Unicorn: Runtime provenance-based detector for advanced persistent threats, in: Network and Distributed System Security Symposium, 2020.
- [7] F. Yang, J. Xu, C. Xiong, et al., Prographer: An anomaly detection system based on provenance graph embedding, in: 32nd USENIX Security Symposium (USENIX Security 23), 2023, pp. 4355-4372.
- [8] L. Moreau, J. Freire, J. Futrelle, R.E. McGrath, P. Paulson, The open provenance model: An overview, in: International Provenance and Annotation Workshop, 2008.
- [9] Abdullellah Alsaheel, Yuhong Nan, Shiqing Ma, Le Yu, Gregory Walkup, Z. Berkay Celik, Xiangyu Zhang, Dongyan Xu, ATLAS: A sequence-based learning approach for attack investigation, in: USENIX Security Symposium, 2021, pp. 3005-3022.
- [10] Carter Yagemann, Mohammad A. Noureddine, Wajih Ul Hassan, Simon Chung, Adam Bates, Wenke Lee, Validating the integrity of audit logs against execution repartitioning attacks, in: Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security, 2021, pp. 3337-3351.
- [11] Peng Fei, Zhou Li, Zhiying Wang, Xiao Yu, Ding Li, Kangkook Jee, Storage-efficient causality analysis on enterprise logs with query-friendly compression, in: USENIX Security Symposium, 2021, pp. 2987-3004.
- [12] Yutao Tang, Ding Li, Zhichun Li, Mu Zhang, Kangkook Jee, Xusheng Xiao, Zhenyu Wu, Junghwan Rhee, Fengyuan Xu, Qun Li, Nodemerger: template based efficient data reduction for big-data causality analysis, in: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, 2018, pp. 1324-1337.
- [13] S.M. Milajerdi, R. Gjomemo, B. Eshete, R. Sekar, V. N.Venkatakrishnan, Holmes: Real-time apt detection through correlation of suspicious information flows, in: 2019 IEEE Symposium on Security and Privacy, 2019.
- [14] W.U. Hassan, A. Bates, D. Marino, Tactical provenance analysis for endpoint detection and response systems, in: IEEE Symposium on Security and Privacy, 2020.
- [15] C. Xiong, et al., Conan: A practical real-time APT detection system with high accuracy and efficiency, IEEE Trans. Dependable Secur. Comput. 19 (1) (2022) 551-565.
- [16] Jun Zengy, Xiang Wang, Jiahao Liu, et al., Shadewatcher: Recommendation-guided cyber threat analysis using system audit records, in: 2022 IEEE Symposium on Security and Privacy, (SP), 2022.
- [17] M. Lv, C. Dong, T. Chen, T. Zhu, Q. Song, Y. Fan, A heterogeneous graph learning model for cyber-attack detection, 2021, CoRR, abs/2112.08986.
- [18] Qi. Wang, Wajih.Ul. Hassan, Ding. Li, et al., You are what you do: Hunting stealthy malware via data provenance analysis, in: NDSS, 2020.
- [19] Su Wang, Zhiliang Wang, Tao Zhou, et al., Threatrace: Detecting and tracing host-based threats in node level through provenance graph learning, in: IEEE Transactions on Information Forensics and Security, 2022.
- [20] Hailun Ding, Juan Zhai, Yuhong Nan, Shiqing Ma, Airtag: Towards automated attack investigation by unsupervised learning with log texts, in: 32nd USENIX Security Symposium (USENIX Security 23), 2023, pp. 373-390.
- [21] M. Rui, Y. Hanbing, W. Qinqin, H. Zhihui, L. Zhuohang, et al., Tdlens: Toward an empirical evaluation of provenance graph-based approach to cyber threat detection, China Commun. 19 (10) (2022) 102-115.
- [22] M. Ur Rehman, H. Ahmadi, W. Ul Hassan, Flash: A comprehensive approach to intrusion detection via provenance graph representation learning, in: IEEE Symposium on Security and Privacy, (SP), San Francisco, CA, USA, 2024, pp. 3552-3570, <http://dx.doi.org/10.1109/SP54263.2024.00139>.
- [23] W. Zhengqiu, Z. Weinuo, Z. Tiantian, D. Zhenhao, S. Haofei, Y. Zhanxiang, T. Ye, et al., Rt-apt: A real-time apt anomaly detection method for large-scale provenance graph, J. Netw. Comput. Appl. (2025) 233.
- [24] S.T. King, P.M. Chen, Backtracking. intrusions, ACM SIGOPS operating systems review, 37 (5) (2003) 223-236.
- [25] S.T. King, Z.M. Mao, D.G. Lucchetti, P.M. Chen, Enriching intrusion alerts through multi-host causality, in: Network and Distributed System Security Symposium, 2005.
- [26] W.U. Hassan, S. Guo, D. Li, Z. Chen, A. Bates, Nodoe: Combatting threat alert fatigue with automated provenance triage, in: Network and Distributed System Security Symposium, 2019.
- [27] R. Yang, S. Ma, H. Xu, X. Zhang, Y. Chen, Uiscope: Accurate, instrumentation-free, and visible attack investigation for gui applications, in: Network and Distributed System Security Symposium, 2020.
- [28] S. Ma, X. Zhang, D. Xu, ProTracer: Towards practical provenance tracing by alternating between logging and tainting, in: Internet Society, 2016.
- [29] Liu.Y.Mu.Z.Ding.L.Jee. K. Mittal. P. Towards a timely causality analysis for enterprise security, in: Network and Distributed System Security Symposium, 2018.
- [30] L. Yu, S. Ma, Z. Zhang, G. Tao, A. Gehani, Alchemist: Fusing application and audit logs for precise attack provenance without instrumentation, in: Network and Distributed System Security Symposium, 2021.
- [31] Kyu Hyung Lee, Xiangyu Zhang, Dongyan Xu, Kyu hyung lee xiangyu zhang dongyan xu loggc: garbage collecting audit log, in: Proceedings of the 2013 ACM SIGSAC Conference on Computer & Communications Security, 2013, pp. 1005-1016.
- [32] N. Hossain, J. Wang, R. Sekar, S.D. Stoller, Dependence-preserving data compaction for scalable forensic analysis, in: USENIX Security Symposium, 2018, pp. 1723-1740.
- [33] Liu Z, Zhou J, Introduction to graph neural networks, Synth. Lect. Artif. Intell. Mach. Learn. 14 (2020) 1-127.
- [34] A.S. Zamzam, N.D. Sidiropoulos, Physics-aware neural networks for distribution system state estimation, vol. 35, 2020, pp. 4347-4356.
- [35] Zhang X, Zitnik M, GnnGuard: Defending graph neural networks against adversarial attacks, 2020.
- [36] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Lio, Y. Bengio, Graph attention networks, in: International Conference on Learning Representations, 2017, pp. 1-12.
- [37] X. Wang, H. Ji, C. Shi, B. Wang, P. Cui, P. Yu, Y. Ye, Heterogeneous graph attention network, in: The World Wide Web Conference, 2019, pp. 2022-2032.
- [38] C. Zhang, D. Song, C. Huang, Ananthram. Swami, Nitesh V. Chawla, Chawla heterogeneous graph neural network, in: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2019, pp. 793-803.
- [39] S. Hochreiter, J. Schmidhuber, Long short-term memory, Neural Comput. 9 (1997) 1735-1780.
- [40] Kiran-Kumar Muniswamy-Reddy, David A. Holland, Uri Braun, Margo Seltzer, Provenance-aware storage systems, in: USENIX Association, 2006.
- [41] A. Gehani, D. Tariq, SPADE: Support for provenance auditing in distributed environments, in: ACM/IFIP/USENIX International Conference on Distributed Systems Platforms and Open Distributed Processing, 2012, pp. 101-120.
- [42] T. Pasquier, X. Han, M. Goldstein, T. Moyer, J. Bacon, Practical whole-system provenance capture, in: Proceedings of the 2017 Symposium on Cloud Computing, 2017, pp. 405-418.
- [43] Wu.Z.Pan.S.Chen.F.Long.G.Zhang. C. Yu.P. S, A comprehensive survey on graph neural networks, in: IEEE Transactions on Neural Networks and Learning Systems, 2019.
- [44] Jian Cui, Kwanwoo Kim, Seung Ho Na, Seungwon Shin, Hetero-SCAN: Towards social context aware fake news detection via heterogeneous graph, Neural Netw. (2021) CoRR, abs/2109.08022.
- [45] Fucheng Liu, Yu Wen, Dongxue Zhang, Xihe Jiang, Xinyu Xing, Dan Meng, Log2vec: A heterogeneous graph embedding based approach for detecting cyber threats within enterprise, in: Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, 2019, pp. 1777-1794.
- [46] M. van Opstal, W. Arbaugh, Operationally Transparent cyber (OpTC) data release, 2019, <https://github.com/FiveDirections/OpTC-data>.
- [47] N. Buduma, N. Locascio, Fundamentals of Deep Learning: Designing Next-Generation Machine Intelligence Algorithms, O'Reilly Media, Inc, 2017.
- [48] Z. Cheng, Q. Lv, J. Liang, Y. Wang, D. Sun, T. Pasquier, X. Han, KAIROS: Practical intrusion detection and investigation using whole-system provenance, in: IEEE Symposium on Security and Privacy, SP, 2023.

- [49] H. Irshad, G. Ciocarlie, A. Gehani, V. Yegneswaran, K.H. Lee, J. Patel, S. Jha, Y. Kwon, D. Xu, X. Zhang, Trace: Enterprise-wide provenance tracking for real-time apt detection, in: IEEE Transactions on Information Forensics and Security, 2021.
- [50] J. Goodfellow, I.J. Shlens, C. Szegedy, Explaining and harnessing adversarial examples, 2015, CoRR, abs/1412.6572.
- [51] Jan.Hendrik. Metzen, Tim. Genewein, Volker. Fischer, Bastian. Bischoff, On detecting adversarial perturbations, in: 5th International Conference on Learning Representations, 2017.
- [52] W. Jin, Y. Ma, X. Liu, X. Tang, S. Wang, J. Tang, Graph structure learning for robust graph neural networks, 2020, pp. 66–74, CoRR, abs/2005.10203.
- [53] T. Ma, C. Xiao, J. Zhou, F. Wang, Drug similarity integration through attentive multi-view graph auto-encoders, 2018, CoRR, abs/1804.10850.
- [54] W. Jin, Y. Li, H. Xu, Y. Wang, J. Tang, Adversarial attacks and defenses on graphs: a review and empirical study, 2020, CoRR, abs/2003.00653.



Lin Wu received her M.E. degree from Guangxi Normal University, in 2020. She is currently pursuing her Ph.D. degree in Huazhong University of Science and Technology (HUST). Her research interests are digital provenance and intrusion detection.



Yulai Xie received the B.E. and Ph.D. degrees in Computer Science from Huazhong University of Science and Technology (HUST), China, in 2007 and 2013, respectively. He was a visiting scholar at the University of California, Santa Cruz in 2010 and a visiting scholar at the Chinese University of Hong Kong in 2015. He is now a professor in School of Cyber Science and Engineering, in HUST, China. His research interests mainly include cloud storage and virtualization, digital provenance, intrusion detection, machine learning and computer architecture.



Shixun Zhao received master's degree from Huazhong University of Science and Technology in 2021. His research interests mainly include digital provenance and intrusion detection.



Pan Zhou received the B.S. degree in the Advanced Class and the M.S. degree from the Department of Electronics and Information Engineering from Huazhong University of Science and Technology (HUST), Wuhan, China, in 2006 and 2008, respectively, and the Ph.D. degree from the School of Electrical and Computer Engineering, Georgia Institute of Technology (Georgia Tech), Atlanta, GA, USA, in 2011. He was a Senior Technical Member at Oracle Inc., America, from 2011 to 2013. He is currently a Professor with the Hubei Engineering Research Center on Big Data Security, School of Cyber Science and Engineering, HUST. His current research interests include security and privacy, big data analytics and machine learning, and information networks.



Dan Feng received her B.E, M.E. and Ph.D. degrees in Computer Science and Technology from Huazhong University of Science and Technology (HUST), China, in 1991, 1994 and 1997 respectively. She is a professor and director of Data Storage System Division, Wuhan National Lab for Optoelectronics. She is also dean of the School of Computer Science and Technology, HUST. Her research interests include computer architecture, massive storage systems, parallel file systems, disk array and solid-state disk. She has over 100 publications in journals and international conferences, including FAST, USENIX ATC, ICDCS, HPDC, SC, ICS and IPDPS. Dr. Feng is a member of IEEE and a member of ACM.



Avani Wildani received her B.S. degree in Computer Science and Mathematics at Harvey Mudd College and her Ph.D. in Computer Science at UC Santa Cruz. She is now an assistant professor in the Department of Computer Science, in Emory University, US. Her interests mainly include information storage and retrieval across different storage models, with application domains including access prediction, data deduplication, archival economics, power management, wireless mesh networks, auditory receptive field characterization, and pollution monitoring.



Yafeng Wu received master's degree in computer science from Huazhong University of Science and Technology in 2019. He is currently pursuing his Ph.D. degree in Huazhong University of Science and Technology (HUST). His research interests mainly include digital provenance and intrusion detection.